

2. SotilityInsurance.sol (continued)

Plain Text

```
struct Policy {
    address holder;
    uint256 coverageAmount;
    uint256 premium;
    uint256 startTime;
    uint256 endTime;
    PolicyStatus status;
}

struct Claim {
    uint256 policyId;
    string reason;
    uint256 requestedAmount;
    uint256 approvedAmount;
    uint256 submissionTime;
    ClaimStatus status;
}

// Insurance token (could be a stablecoin)
IERC20 public insuranceToken;

// Governance token for possible premium discounts
IERC20 public governanceToken;

// Policy and claims data
mapping(uint256 => Policy) public policies;
mapping(uint256 => Claim) public claims;
uint256 public policyCount;
uint256 public claimCount;

// Insurance fund balance
uint256 public totalCoverageProvided;
uint256 public totalPremiumsCollected;
uint256 public totalClaimsPaid;

// Premium rates (basis points, 100 = 1%)
uint256 public basePremiumRate = 500; // 5% default

event PolicyCreated(uint256 indexed policyId, address indexed holder,
uint256 coverageAmount);
event PolicyCancelled(uint256 indexed policyId);
event ClaimSubmitted(uint256 indexed claimId, uint256 indexed policyId,
uint256 requestedAmount);
event ClaimProcessed(uint256 indexed claimId, ClaimStatus status,
```

```

uint256 approvedAmount);
    event PremiumRateUpdated(uint256 newRate);

    constructor(address _insuranceToken, address _governanceToken) {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(CLAIM_APPROVER_ROLE, msg.sender);
        _grantRole(PREMIUM_ADJUSTER_ROLE, msg.sender);

        insuranceToken = IERC20(_insuranceToken);
        governanceToken = IERC20(_governanceToken);
    }

    /**
     * @dev Get a quote for insurance coverage
     * @param coverageAmount Amount of coverage desired
     * @param durationDays Duration of coverage in days
     */
    function getQuote(uint256 coverageAmount, uint256 durationDays) public
    view returns (uint256) {
        uint256 annualPremium = (coverageAmount * basePremiumRate) / 10000;
        uint256 dailyPremium = annualPremium / 365;

        // Apply governance token discount if applicable
        uint256 governanceBalance = governanceToken.balanceOf(msg.sender);
        uint256 discountRate = _calculateDiscountRate(governanceBalance);

        uint256 totalPremium = (dailyPremium * durationDays);
        if (discountRate > 0) {
            totalPremium = totalPremium * (10000 - discountRate) / 10000;
        }

        return totalPremium;
    }

    /**
     * @dev Purchase an insurance policy
     * @param coverageAmount Amount of coverage desired
     * @param durationDays Duration of coverage in days
     */
    function purchasePolicy(uint256 coverageAmount, uint256 durationDays)
    external nonReentrant returns (uint256) {
        require(coverageAmount > 0, "Coverage must be greater than 0");
        require(durationDays > 0 && durationDays <= 365, "Invalid duration");

        uint256 premium = getQuote(coverageAmount, durationDays);
        require(insuranceToken.transferFrom(msg.sender, address(this),
        premium), "Premium payment failed");
    }

```

```

    policyCount++;
    uint256 endTime = block.timestamp + (durationDays * 1 days);

    policies[policyCount] = Policy({
        holder: msg.sender,
        coverageAmount: coverageAmount,
        premium: premium,
        startTime: block.timestamp,
        endTime: endTime,
        status: PolicyStatus.Active
    });

    totalCoverageProvided += coverageAmount;
    totalPremiumsCollected += premium;

    emit PolicyCreated(policyCount, msg.sender, coverageAmount);

    return policyCount;
}

/**
 * @dev Submit a claim on an active policy
 * @param policyId ID of the policy
 * @param amount Amount requested (must be <= coverage amount)
 * @param reason Reason for the claim
 */
function submitClaim(uint256 policyId, uint256 amount, string calldata
reason) external nonReentrant returns (uint256) {
    require(policyId <= policyCount, "Policy does not exist");
    Policy storage policy = policies[policyId];

    require(policy.holder == msg.sender, "Not the policy holder");
    require(policy.status == PolicyStatus.Active, "Policy not active");
    require(block.timestamp < policy.endTime, "Policy expired");
    require(amount <= policy.coverageAmount, "Amount exceeds coverage");

    claimCount++;

    claims[claimCount] = Claim({
        policyId: policyId,
        reason: reason,
        requestedAmount: amount,
        approvedAmount: 0,
        submissionTime: block.timestamp,
        status: ClaimStatus.Pending
    });

    emit ClaimSubmitted(claimCount, policyId, amount);

```

```

        return claimCount;
    }

    /**
     * @dev Process a pending claim (approve or reject)
     * @param claimId ID of the claim
     * @param approved Whether the claim is approved
     * @param approvedAmount Amount approved (if approved)
     */
    function processClaim(uint256 claimId, bool approved, uint256
approvedAmount) external onlyRole(CLAIM_APPROVER_ROLE) nonReentrant {
        require(claimId <= claimCount, "Claim does not exist");

        Claim storage claim = claims[claimId];
        Policy storage policy = policies[claim.policyId];

        require(claim.status == ClaimStatus.Pending, "Claim not pending");
        require(policy.status == PolicyStatus.Active, "Policy not active");

        if (approved) {
            require(approvedAmount <= claim.requestedAmount, "Approved
amount too high");
            require(approvedAmount <= policy.coverageAmount, "Approved
amount exceeds coverage");

            claim.status = ClaimStatus.Approved;
            claim.approvedAmount = approvedAmount;
            policy.status = PolicyStatus.Claimed;

            // Transfer approved amount to claimant
            require(insuranceToken.transfer(policy.holder, approvedAmount),
"Claim payment failed");

            totalClaimsPaid += approvedAmount;
            totalCoverageProvided -= policy.coverageAmount;
        } else {
            claim.status = ClaimStatus.Rejected;
        }

        emit ClaimProcessed(claimId, claim.status, claim.approvedAmount);
    }

    /**
     * @dev Update the base premium rate
     * @param newRate New premium rate in basis points
     */
    function updatePremiumRate(uint256 newRate) external

```

```

onlyRole(PREMIUM_ADJUSTER_ROLE) {
    require(newRate <= 2000, "Rate too high"); // Max 20%
    basePremiumRate = newRate;
    emit PremiumRateUpdated(newRate);
}

/**
 * @dev Calculate discount rate based on governance token holdings
 * @param governanceBalance Balance of governance tokens
 */
function _calculateDiscountRate(uint256 governanceBalance) internal pure
returns (uint256) {
    // Example tiered discount:
    // 0-999 tokens: 0% discount
    // 1000-9999 tokens: 5% discount
    // 10000+ tokens: 10% discount

    if (governanceBalance >= 10000 * 1e18) {
        return 1000; // 10% discount
    } else if (governanceBalance >= 1000 * 1e18) {
        return 500; // 5% discount
    } else {
        return 0; // No discount
    }
}

/**
 * @dev Get policy details
 * @param policyId ID of the policy
 */
function getPolicyDetails(uint256 policyId) external view returns (
    address holder,
    uint256 coverageAmount,
    uint256 premium,
    uint256 startTime,
    uint256 endTime,
    PolicyStatus status
) {
    require(policyId <= policyCount, "Policy does not exist");
    Policy storage policy = policies[policyId];

    return (
        policy.holder,
        policy.coverageAmount,
        policy.premium,
        policy.startTime,
        policy.endTime,
        policy.status
    )
}

```

```

    );
}

/**
 * @dev Get claim details
 * @param claimId ID of the claim
 */
function getClaimDetails(uint256 claimId) external view returns (
    uint256 policyId,
    string memory reason,
    uint256 requestedAmount,
    uint256 approvedAmount,
    uint256 submissionTime,
    ClaimStatus status
) {
    require(claimId <= claimCount, "Claim does not exist");
    Claim storage claim = claims[claimId];

    return (
        claim.policyId,
        claim.reason,
        claim.requestedAmount,
        claim.approvedAmount,
        claim.submissionTime,
        claim.status
    );
}

/**
 * @dev Get insurance fund statistics
 */
function getFundStatistics() external view returns (
    uint256 activeCoverage,
    uint256 premiumsCollected,
    uint256 claimsPaid,
    uint256 availableFunds
) {
    return (
        totalCoverageProvided,
        totalPremiumsCollected,
        totalClaimsPaid,
        insuranceToken.balanceOf(address(this))
    );
}
}

```

Deployment and Integration Strategy

1. Deployment Flow

The Sotility ecosystem should be deployed in a specific order to ensure proper dependencies and permissions:

1. Deploy Core Tokens:

- SotilityStableToken (SST)
- SotilityOwnershipToken (SOT)
- SoGoodUtilityGovernance (SUG)

2. Deploy Infrastructure Contracts:

- SotilityEmergencyShutdown (with initial guardians)
- AIOracleManager
- MultiOracleAggregator

3. Deploy Financial Contracts:

- SotilityTreasuryRouter (with initial pool addresses)
- SotilityExchange (with SST address)
- SotilityYieldEngine and VaultFactory

4. Deploy Identity Contracts:

- SotilityBadgeNFT
- SotilityProfileRegistry
- SotilityZKIdentity
- SotilityProofOfPersonhood

5. Deploy Social Layer:

- SoGoodFeed
- SotilityContentGovernance

6. Deploy Governance:

- ProposalManager
- SoGoodDAOFactory

7. Deploy Advanced Mechanisms:

- SotilityVeToken (with SOT as base token)
- SotilityLiquidStaking (with SOT as staking token)

- SotilityBridgeAdapter (for each token)
- SotilityAdaptiveTokenomics
- SotilityInsurance

2. Role Assignment Strategy

1. Initial Admin Setup:

- Deploy all contracts with a secure multi-sig wallet as DEFAULT_ADMIN_ROLE
- Assign GUARDIAN_ROLE in EmergencyShutdown to separate trusted entities

2. Role Distribution:

- Grant MINTER_ROLE for SST to SotilityExchange
- Grant VERIFIER_ROLE in SoGoodFeed to AIOracleManager
- Connect MultiOracleAggregator with AI oracle agents
- Set up cross-contract permissions

3. Progressive Decentralization:

- Transfer admin roles to DAO governance after thorough testing
- Implement timelocks for sensitive operations
- Gradually expand guardian committee

3. Testing and Auditing Strategy

1. Comprehensive Testing Suite:

- Unit tests for each contract
- Integration tests for cross-contract interactions
- Fuzzing/property-based testing for complex logic
- Formal verification for critical components

2. Staged Auditing Process:

- Internal peer reviews
- External audit by a reputable firm
- Bug bounty program
- Public open-source review period

3. Simulation and Stress Testing:

- Economic attack simulations
- AI agent behavior simulations

- Extreme market condition modeling
- Cross-chain communication testing

4. Monitoring and Maintenance

1. On-Chain Monitoring:

- Implement event-based alerting system
- Set up multi-oracle monitoring for anomaly detection
- Create dashboards for key protocol metrics

2. Continuous Improvement:

- Regular security reviews
- Implementation of governance-approved upgrades
- Adaptation to evolving ecosystem standards

3. Disaster Recovery:

- Clear procedures for different emergency scenarios
- Regular drills for emergency response team
- Strategic reserves for contingency funding

Optional Extension Contracts

For future development and enhanced functionality, the following contracts could be considered:

1. **SotilityLiquidityIncentives:** To incentivize liquidity provision for tokens
2. **SotilityFuturesMarket:** For revenue-backed derivatives
3. **SotilityAutonomousDAO:** For AI-augmented DAO governance
4. **SotilityVirtualIdentity:** For metaverse/virtual representations tied to on-chain identity
5. **SotilityCreatorRoyalties:** For tracking and enforcing royalties on content

These extensions would further strengthen the ecosystem's capabilities and provide additional utility for participants.